Karadeniz Teknik Üniversitesi

Fen Fakültesi – Bilgisayar Bilimleri Bölümü

Metin, UI ve Menü Sistemleri

PyGame ile Oyun Geliştirme – Hafta 9

Dr. Tolga BERBER

Bu Hafta Ne Öğreneceğiz?

1. Metin Render Etme
2. UI Elemanları
3. Buton Sınıfı
4. Menü Sistemleri

Metin Render Etme

PyGame Font Sistemi

- `pygame.font` modülü
- İki yöntem:
  - `Font(dosya, boyut)` – dosyadan yükle
  - `SysFont(isim, boyut)` – sistemden yükle
- `None` = varsayılan font

Font Oluşturma

```
font = Font(None, 36)
arial = SysFont("arial", 24)
```

render() ve blit()

Metin Render Akışı

Font → render() → Surface → blit() → Ekran

```
font = pygame.font.Font(None, 48)
metin = font.render("Merhaba!", True, (255, 255, 0))
ekran.blit(metin, (250, 280))
```

render() Parametreleri

`render(metin, antialias, renk, arkaplan=None)`

Metin Hizalama

```
# Ortaya hizalama
metin = font.render("Orta", True, (255,255,255))
rect = metin.get_rect(center=(400, 300))
ekran.blit(metin, rect)

# Saga hizalama
rect = metin.get_rect(topright=(790, 10))
```

Hizalama Seçenekleri

`center`, `topleft`, `topright`
`midtop`, `midbottom`
`midleft`, `midright`

Çok Satırlı

`split("\n")` ile böl
Her satırı ayrı render et
`get_linesize()` kullan

Dinamik Metin

```
# f-string ile dinamik metin
skor_metin = font.render(f"Skor: {skor}", True, (255,215,0))

# FPS gostergesi
fps = saat.get_fps()
fps_metin = font.render(f"FPS: {fps:.0f}", True, (0,255,0))

# Geri sayim
kalan = max(0, sure_limiti - gecen_saniye)
sure = font.render(f"{kalan//60:02d}:{kalan%60:02d}",
                   True, (255,255,255))
```

Performans

Değişmeyen metinleri döngü dışında render et, sadece blit() çağır!

UI Elemanları

Skor Göstergesi

```python
class SkorGostergesi:
    def __init__(self, x, y, font_boyutu=32):
        self.skor = 0
        self.font = pygame.font.Font(None, font_boyutu)
        self._guncelle_yuzeyi()

    def _guncelle_yuzeyi(self):
        self.yuzeyi = self.font.render(
            f"Skor: {self.skor:06d}", True, (255,215,0))

    def arttir(self, miktar=1):
        self.skor += miktar
        self._guncelle_yuzeyi()

    def ciz(self, ekran):
        ekran.blit(self.yuzeyi, (self.x, self.y))
```

Sağlık Barı

```
def saglik_bari_ciz(ekran, x, y, gen, yuk, can, max_can):
    oran = can / max_can
    doluluk = int(gen * oran)

    if oran > 0.6:   renk = (0, 200, 0)      # yesil
    elif oran > 0.3: renk = (255, 200, 0)     # sari
    else:            renk = (200, 0, 0)       # kirmizi

    pygame.draw.rect(ekran, (60,60,60), (x,y,gen,yuk))
    pygame.draw.rect(ekran, renk, (x, y, doluluk, yuk))
    pygame.draw.rect(ekran, (200,200,200), (x,y,gen,yuk), 2)
```

Renk Geçişi

Can $>$ 60%: yeşil | Can 30–60%: sarı | Can $<$ 30%: kırmızı

HUD Tasarımı

- **HUD** = Heads-Up Display
- Oyun bilgilerini gösteren katman
- Bileşenler:
  - Sağlık barı (sol üst)
  - Skor (sağ üst)
  - Seviye (orta üst)
  - FPS (debug)
- Her zaman en son çizilir!

Yarı Şeffaf Panel

```
panel = Surface((g, y))
panel.set_alpha(150)
panel.fill((0, 0, 0))
ekran.blit(panel, (0, 0))
```

Buton Sınıfı

Temel Buton Yapısı

```
class Buton:
    def __init__(self, x, y, gen, yuk, metin, callback=None):
        self.rect = pygame.Rect(x, y, gen, yuk)
        self.metin = metin
        self.callback = callback
        self.uzerinde = False  # hover durumu

    def ciz(self, ekran):
        renk = self.hover_renk if self.uzerinde else self.renk
        pygame.draw.rect(ekran, renk, self.rect, border_radius=5)
        # Metni ortala
        metin_y = self.font.render(self.metin, True, (255,255,255))
        rect = metin_y.get_rect(center=self.rect.center)
        ekran.blit(metin_y, rect)
```

Hover ve Click

Hover Algılama

```
def guncelle(self):
    fare = pygame.mouse.get_pos()
    self.uzerinde = \
        self.rect.collidepoint(fare)
```

Click Algılama

```
def olay_isle(self, olay):
    if olay.type == MOUSEBUTTONDOWN:
        if olay.button == 1:
            if self.rect.collidepoint(
                    olay.pos):
                if self.callback:
                    self.callback()
```

Callback Mekanizması

`callback=oyunu_baslat` → Fonksiyon referansı (parantez yok!)

Menü Sistemleri

Oyun Durumu Yönetimi

Durumlar

- MENU – Ana menü ekranı
- OYUN – Oyun ekranı
- PAUSE – Duraklatma
- GAME_OVER – Bitiş ekranı

Geçişler

Menü → Oyun (Başla)
Oyun → Pause (ESC)
Pause → Oyun (Devam)
Oyun → Game Over (can=0)
Game Over → Oyun (Tekrar)
Game Over → Menü

Durum Tabanlı Ana Döngü

```
while calistir:
    olaylar = pygame.event.get()

    if durum == MENU:
        menu_isle(olaylar)
        menu_ciz(ekran)
    elif durum == OYUN:
        oyun_isle(olaylar)
        oyun_ciz(ekran)
    elif durum == PAUSE:
        pause_isle(olaylar)
        pause_ciz(ekran)
    elif durum == GAME_OVER:
        gameover_isle(olaylar)
        gameover_ciz(ekran)

    pygame.display.flip()
```

Pause Menüsü

1. ESC tuşu ile pause'a geç
2. `ekran.copy()` ile arka planı yakala
3. Yarı şeffaf karartma çiz
4. Butonları göster
5. "Devam Et" ile oyuna dön

Dikkat!

Pause sırasında:

- Oyun güncellenmez
- Zamanlayıcılar duraklatılmalı
- Arka plan donmuş olarak görünür

Mini Proje: Tam Menü Sistemi

Proje Bileşenleri

- **Ana Menü**: Başlık + Butonlar (Başla, Çıkış)
- **Oyun Ekranı**: HUD + Oyun mekaniği
- **Pause Menüsü**: Devam / Ana Menü / Çıkış
- **Game Over**: Skor gösterimi + Tekrar Oyna

Tasarım Deseni: State Machine

Her durumun kendi `olay_isle()`, `guncelle()` ve `ciz()` metotları var. Merkezi yönetici durumlar arası geçişi kontrol eder.

Özet

Bu Hafta Öğrendiklerimiz

- Font sistemi ve metin render
- Skor ve sağlık barı
- HUD tasarımı
- Buton sınıfı (hover + click)
- Menü sistemleri
- State machine deseni

Gelecek Hafta

- Tile-based oyun dünyası
- Tile harita editörü
- Kamera sistemi
- Platformer temelleri